CS246 Spring 2026 Project Deliverables

Due Date 1: Monday, July 20, 2026, 5:00 pm
Due Date 2: Tuesday, July 28, 2026, 5:00 pm

What happens if you declare a 48-hour absence on the project, given that it is a group project? Under no circumstances will the project be excused or extended as a result of illness or declared absence. As long as the project is handed in, it will be marked as is. The only circumstance under which a project may not be handed in is if ALL group members declare an absence (a special case of this would be that you did the project yourself). In that case, the accommodation is an INC in the course, and you will have to do the project in a future term. Note that INCs are never automatic, even in this situation. An INC is only awarded if, based on your performance in the rest of the term (including, in this case, the final exam), instructors feel you had a strong probability of passing the course. If you are ill for the project demo day, you can be excused from attending the demo, provided that your other group members attend.

Your project will be graded based on correctness and completeness (60%), documentation (20%), and design (20%). Grading will proceed in two phases, outlined below:

Correctness and Completeness

The correctness and completeness component of your project will be assessed via a live demo in front of a TA. You will need to sign up for a 30-minute demo slot with a TA; all group members should show up, and you should plan a demo that takes the TA on a tour through all of the required features of the project. Be prepared to explain to the TA how you implemented the various parts of your project. At any time, the TA may ask you to demonstrate a particular element of your project (e.g., whether some boundary case is handled properly). If there is a component of your project that is not working, advise the TA, rather than pretend that it does.

After you have demonstrated the core requirements, you may demonstrate any additional features of the project that you may have implemented, for extra credit. The TA will assess the worth of your additional features after your demo is over.

If your project submission on Marmoset does not compile, you will not be allowed to continue with your demo. The TA will not, under any circumstance, change your code, or allow you to change your code, in order to make your submission compile. You will receive a grade of 0 on the correctness and completeness portion of the project.

Documentation and Design

The design of your project is an important part of your overall assessment. For top grades, it is not enough that your program simply work; it must demonstrate a solid object-oriented design. Thus, you will need to spend considerable time planning out your classes and their interactions, aiming to minimize coupling while maximizing cohesion. You must plan for change in your program. Imagine that the specification will change on the day before the final due date (it won't, but imagine it will). How easily can you accommodate change?

As part of your design, you should anticipate various ways your project could change. Perhaps a change in rules, or input syntax, or a whole new feature—who knows? Plan all of your project features to accommodate change, with minimal modification to your original program, and minimal recompilation. In your final design document, outline the ways in which your design accommodates new features and changes to existing features. Be specific. You may wish to implement some of these for extra credit. **This discussion is a very important component of your design grade; if you spend less than a page on it, you probably aren't saying enough.**

Your documentation and design will be assessed via written documents that you will submit to Marmoset as PDFs.

Extra Credit Features

All of the available projects give you the opportunity to earn up to 10% extra credit for enhancements to the core projects. Keep the following rules and guidelines in mind when planning enhancements:

- It is far more important to have your core project working well, than to have a poorly done project with extras. Remember, the enhancements are only worth 10%. **Your documentation and design are also more valuable than your enhancements, so please take time to do these well.**

- Your program must be able to run the core project, without enhancements, as well as with them. You are not allowed to submit two programs for grading, one with enhancements and one without. You are also not allowed to recompile your program with different compiler flags to produce versions with and without enhancements. You may select or deselect your enhancements via flag arguments on the command line. (E.g. `./project -enablebonus`, or something similar.) Even better would be if you could turn your enhancements on and off dynamically, as the program is running.
- Markers will not assign values to individual bonus features during the demo. You may show what you have done, and the TA will take notes, but values will be decided once all demos are completed.
- One enhancement that is available for all three projects is offered as a challenge: complete the entire project, without leaks, and without explicitly managing your own memory. In other words, handle all memory management via STL containers and smart pointers. If you do this, there should be no `delete`[1] statements in your program at all, and very few raw pointers (the only raw pointers you would be permitted to have are those that are not meant to express ownership). Successful completion of this challenge will earn you 4 of the available 10 bonus marks. If your program leaks, these marks cannot be earned. Also note that these 4 marks are not reserved for this feature; it is theoretically possible to get all 10 bonus marks without implementing this challenge.

On Due Date 1

Submit a plan of attack. This should include a UML class model that describes how you anticipate your system to be structured. **Note:** even if you complete the project by due date 1, the UML class model you submit must be one that you built prior to starting the project. (You may find the UML Reference Card, contained in `handouts/UMLReferenceCard.pdf` helpful.)

Your UML class model should show the classes that make up your project and the relationships between them. You only need to show public methods (i.e., you can leave out private fields and protected/private methods, unless you need to show them to illustrate a point, e.g., a design pattern). **Do not show the big 5 operations, or any other constructors, accessors, or mutators.** You will not be graded on the degree to which you adhere to this model, but you will be asked to account for any differences that arise between this model and your final submission.

File to Submit: `uml.pdf`.

In addition, your plan of attack must include a breakdown of the project, indicating what you plan to do first, what will come next, and so on. Include estimated completion dates, and which partner will be responsible for which parts of the project. You should try to stick to your plan, but you will not be graded by the degree to which you stick to it. Your initial plan should be realistic, and you will be expected to explain why you had to deviate from your plan (if you did).

Finally, your initial plan of attack must include answers to all questions listed within the project specification itself. You should answer in terms of how you would anticipate solving these problems in your project, even though you are not strictly required to do so. If your answers turn out to be inconsistent with your final design, you will have an opportunity to submit revised answers on Due Date 2.

File to Submit: `plan.pdf`

Your plan should be no more than 5 double-sided, single-spaced pages long.

On Due Date 2

On Due Date 2, you must submit all of your code to Marmoset, together with either a `Makefile` (creates the executable given the command `make`) or files `order.txt` and `syslibs.txt` (respectively contains the list of your files in dependency order and the list of system library files imported; for use with the `compile` bash script in the `tools` folder of your repository). **You should be using modules unless you are using C++ libraries that have not been converted to C++20 modules.** Your executable should be called `constructor` for Constructor, `sorcery` for Sorcery, or `cc3k` for CC3K.

In addition, you must submit a final design document. It must outline the final, actual design of your project, and how it differed from your design on Due Date 1 (if it did). You should include an updated UML class model, reflecting the actual structure of your project. Do this even if it is the same as the original UML class model.

Your document should provide an overview of all aspects of your project, including how, at a high level, they were implemented. If you made use of design patterns, clearly indicate where.

Your system should employ good object-oriented design techniques, as presented in class.

[1] Don't use the word "delete" in your comments either since Marmoset uses `egrep` to search for the keyword when determining if your project is eligible for the bonus.

Include all answers to questions posed within the project specification, and indicate how they differ from the answers you gave on Due Date 1 (if they did).

You should not expect the TA to read through all of your code. Therefore, your design document should stand alone—the TA should not need to have your code open in order to understand your document. However, if you wish to highlight certain aspects of your design, you should indicate clearly where they can be found in your code, if the TA wants to have a look.

The most important aspect of your design document is a discussion of how your chosen design accommodates change, as described earlier in this document. You should also discuss the cohesion and coupling of your chosen program modules. Most of the design portion of your grade will be based on this summary.

Finally, answer the following questions:

1. What lessons did this project teach you about developing software in teams? If you worked alone, what lessons did you learn about writing large programs?

2. What would you have done differently if you had the chance to start over?

Files to Submit: `uml-final.pdf`, `design.pdf`

Length and Format of Document

We expect that your final document will be 5 – 10 pages long, where each page has a word density similar to what you see in this guideline document.

Organize your document into sections, using the following headings:

- Introduction (if needed)
- Overview (describe the overall structure of your project)
- Design (describe the specific techniques you used to solve the various design challenges in the project)
- Resilience to Change (describe how your design supports the possibility of various changes to the program specification)
- Answers to Questions (the ones in your project specification)
- Extra Credit Features (what you did, why they were challenging, how you solved them—if necessary)
- Final Questions (the last two questions in this document).
- Conclusion (if needed)

The document you submit will be used for both your documentation mark and your design mark. Your design will be assessed according to how you solved the problem, as described in your document, and in your Due Date 1 UML class model. Your documentation will be graded according to how well you described your project, how well you described your design (i.e., is it clear and well-communicated?), and your Due Date 1 document. **Together, these account for 40% of your grade, so these documents are very important.** The Due Date 2 document is worth much more than the Due Date 1 document.

Late Policy

We **strongly** discourage you from making last-minute changes to your code. The risk of your code not compiling, or otherwise not working, is too high. We recommend writing no new code in the last six hours before the code is due. Save those hours for preparing for your demo, and for emergency bug fixes.

Late submissions will not be accepted.